LangChain Lab 2: The "No-Scrolling" Cheat Sheet

This is the distilled, interview-ready version of the lab. Read this instead of scrolling through the boilerplate code.

[SKIP] The SKIP Pile (Basic/Legacy Concepts)

Exercise 1: Model Temperatures

- Concept: Temperature controls "creativity". 0.1 = Strict/Factual. 0.8 = Creative/Random. You already know this.

Exercise 2: Output Parsers (JSON)

- Concept: Forcing the LLM to reply in a specific format (like JSON or CSV).
- Why skip? The lab uses string instructions to beg the LLM for JSON. This is the old way. You are already using Pydantic in your live project, which is the modern, bulletproof way to do this.

Exercise 5: Chatbot Memory

- Concept: LLMs have absolutely zero memory. They are stateless.
- How it works: To make a chatbot, you have to save every single message (Human and AI) in a list (database). Every time you send a new message, the system injects the entire past conversation history into the prompt behind the scenes so the LLM can re-read it.

[MUST KNOW] The STRUGGLE Pile (Core Interview Knowledge)

You must understand these concepts deeply for technical interviews.

Exercise 3: Document Loaders & Splitters (The Data Prep)

Before you can search a document, you have to prepare it.

1. Loaders: Tools (like PyPDFLoader or WebBaseLoader) that ingest raw files and turn them into LangChain Document objects (which hold the raw text + metadata like the URL or page number).
2. Splitters: You can't feed a 500-page PDF to an LLM. You must chop it into chunks.

- Interview Question: What is the difference between CharacterTextSplitter and RecursiveCharacterTextSplitter?
- Answer: Character just blindly chops text every 500 characters, which might slice a sentence in half. Recursive is smart: it tries to chop at paragraphs first, then sentences, then words, to ensure the chunk actually retains its meaning.

Exercise 4: Vector Stores & Retrievers (The Search Engine)

This is the heart of RAG (Retrieval-Augmented Generation).

1. Embeddings: Translating the chopped-up text chunks into massive arrays of numbers (vectors).
2. Vector Space: A massive 3D graph where chunks with similar meanings are plotted close to each other.
3. Vector Database (Chroma): The system that stores these number arrays.
4. Semantic Search: When a user asks a question, the question is translated into numbers, plotted on the graph, and the database grabs the closest text chunks.

[!TIP]

Advanced Interview Concept: "Small-to-Big Retrieval" (ParentDocumentRetriever)

The lab briefly introduces the ParentDocumentRetriever. This is a massive interview topic!

- The Problem: If chunks are too large, the math gets muddy and semantic search fails. If chunks are too small (just one sentence), the search works great, but when you hand that one sentence to the LLM, it doesn't have enough context to give a good answer.

LangChain Lab 2: Interview Cheat Sheet

- The Solution: You split the document twice. You create tiny chunks (for the search engine to use) and map them to their large "Parent" chunks (for the LLM to read). When the user asks a question, the search engine matches a tiny chunk, but then reaches up and hands the entire large Parent chunk to the LLM.

[!IMPORTANT]

The RAG Pipeline Flow:

Load Doc -> Split into Chunks -> Convert to Numbers (Embed) -> Store in Database (Chroma) -> Retrieve based on meaning.

Exercise 7: Agents & Tools (The "Brain")

This is the most advanced concept in 2026 AI Engineering.

- Chains (Legacy): Hardcoded step-by-step instructions. (Step 1 -> Step 2 -> Step 3).

- Agents (Modern): Non-deterministic. You give the LLM a goal and a toolbox (like a Calculator function or a Web Search function). The LLM acts as the "brain". It looks at the goal, decides on its own which tool to use, executes the tool, observes the result, and decides if it needs to use another tool before answering the user.

- ReAct Framework: The specific thought process the agent is trained to use: Reason (Thought) -> Act (Action/Input) -> Observe (Result).

- Syntax Warning: Learn the ReAct concept above! But don't memorize the lab's AgentExecutor code syntax; in Course 7 you will build ReAct agents using LangGraph instead.

[SKIM] The SKIM Pile

Exercise 6: Multi-Step Processing

This just shows the evolution of LangChain code syntax.

- Old Way (SequentialChain): Very verbose, hard to read, lots of boilerplate class names.

- New Way (LCEL / | pipe operator): Functional and clean. You string tasks together exactly like a bash terminal prompt | llm | output_parser.